

C语言基础：二维数组与字符数组

7.2 二维数组

[!abstract] 什么是二维数组？二维数组可以看作是一个“表格”或“矩阵”，包含行和列。

- 本质上，二维数组是一个**元素为一维数组**的一维数组。

7.2.1 二维数组的定义

语法格式： 类型名 数组名 [行长度] [列长度];

示例：

```
int a[3][2]; // 定义了一个 3 行 2 列的整数数组
```

[!info] 内存结构 二维数组在内存中是**按行连续存放**的。对于 `a[3][2]`，其元素在内存中的排列顺序为：`a[0][0]`，`a[0][1]`，`a[1][0]`，`a[1][1]`，`a[2][0]`，`a[2][1]`

7.2.2 二维数组元素的引用

- **引用格式：** 数组名 [行下标] [列下标]
- **下标范围：**
 - 行下标：0 到 行长度 - 1
 - 列下标：0 到 列长度 - 1

[!warning] 注意 行下标和列下标都必须从 0 开始。

7.2.3 二维数组的初始化

- **分行赋初值：**

```
int a[3][3] = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};
```

- **顺序赋初值**（按照内存排列顺序）：

```
int a[3][3] = {1, 2, 3, 4, 5, 6, 7, 8, 9};
```

- **部分赋初值**（未赋值的元素默认为 0）：

```
int a[3][3] = {{1, 2}, {3}, {4, 5}};
// 结果为:
// 1 2 0
// 3 0 0
// 4 5 0
```

- **省略行长度**（只能省略第一维的长度，不能省略第二维）：

```
int a[][3] = {1, 2, 3, 4, 5, 6, 7}; // 编译器会自动计算出需要 3 行
```

7.2.4 典型示例：找出二维数组中的最大值及其下标

```
#include <stdio.h>
int main(void) {
    int i, j, row, col, max;
    int a[3][2];

    printf("Enter 6 integers: \n");
    for(i = 0; i < 3; i++)
        for(j = 0; j < 2; j++)
            scanf("%d", &a[i][j]); // 输入二维数组

    max = a[0][0]; // 假设第一个元素是最大值
    row = 0;
    col = 0;

    for(i = 0; i < 3; i++)
        for(j = 0; j < 2; j++)
            if(a[i][j] > max){
                max = a[i][j]; // 更新最大值
                row = i;       // 记录最大值所在的行
                col = j;       // 记录最大值所在的列
            }

    printf("max = a[%d][%d] = %d\n", row, col, max);
    return 0;
}
```

7.3 字符数组与字符串

[!abstract] 核心概念

- **字符数组**：用来存放字符数据的数组，每个元素存放一个字符。
- **字符串**：由双引号括起来的字符序列（例如 "Happy"）。
- △ 在 C 语言中，字符串是作为字符数组来处理的。

7.3.1 字符串的表示形式：添加结束符 `\0`

[!important] 字符串的标志 为了判断字符串是否结束，C 语言规定以字符 `'\0'`（ASCII 码为 0，代表空字符）作为字符串的结束标志。

- 如果一个字符数组中包含了 `'\0'`，那么 `'\0'` 前面的所有字符就组成了一个字符串。
- 字符串 `"Happy"` 的实际长度是 5，但在内存中需要占用 **6 个字节**（最后多一个 `'\0'`）。

7.3.2 字符数组的定义与初始化

1. 逐个字符赋值：

```
char s[6] = {'H', 'a', 'p', 'p', 'y', '\0'};
char s[6] = {'H', 'a', 'p', 'p', 'y'}; // 最后一个元素默认初始化为 '\0'
char t[5] = {'H', 'a', 'p', 'p', 'y'}; // 这是一个字符数组，但不能被称为字符串，因为它没有 '\0'
```

2. 使用字符串常量初始化（最常用）：

```
char s[6] = {"Happy"};
char s[6] = "Happy"; // 大括号可以省略
char s[] = "Happy"; // 编译器自动计算大小为 6
```

[!danger] 常见错误 `char s[5] = "Happy"`；这是错误的！因为 `"Happy"` 包含了 6 个字符（5 个字母 + 1 个 `'\0'`），放不进大小为 5 的数组中。

7.3.3 字符串的基本操作

处理字符串时，最常见的做法是使用循环，并以 `'\0'` 作为循环结束的条件。

示例 1：计算字符串的长度 (不包含 `\0`)

```
int len = 0;
while (s[len] != '\0') {
    len++;
}
// 或者使用 for 循环：
for(len = 0; s[len] != '\0'; len++);
```

示例 2：将字符串中的数字字符转换为整数

```
int i, n = 0;
char s[] = "123";
```

```
for(i = 0; s[i] != '\0'; i++) {  
    // 确保当前字符是数字 '0' 到 '9' 之间  
    if(s[i] >= '0' && s[i] <= '9') {  
        n = n * 10 + (s[i] - '0'); // 将字符转换为对应的数字值并累加  
    } else {  
        break; // 遇到非数字字符则停止转换  
    }  
}  
printf("digit = %d\n", n); // 输出结果: 123
```

[!tip] 小技巧：字符转数字 字符 '1' 的 ASCII 码是 49，字符 '0' 的 ASCII 码是 48。因此， $'1' - '0' = 49 - 48 = 1$ 。利用这个特性，可以将任何数字字符转换为其对应的数值。